

CONSERVATOIRE NATIONAL DES ARTS ET METIERS

**Rapport de projet présenté dans le cadre de
l'UE NFE204 « Bases de données documentaires et distribuées »**

par
VANNIER Nicolas

**Etude et expérimentation d'un système
de bases de données NoSQL distribué**

le cnam

Table des matières

TABLE DES MATIERES.....	1
I. DESCRIPTION DU PROJET.....	2
II. LES DONNEES.....	3
1. Fournisseurs et structures des données	3
2. Collecte et volumétrie	6
III. LE SYSTEME NOSQL RIAK	8
1. Présentation de Riak.....	8
2. Mise en œuvre.....	10
a. Installation	10
b. Test des fonctionnalités de base	10
3. Expérimentations & approfondissement.....	12
a. Construction d'un mini-cluster (2 nœuds).....	12
b. Constitution d'une collection de données.....	17
c. Riak Search (Solr).....	17
d. MapReduce avec Riak.....	17
IV. CONCLUSION.....	18
1. Bilan de l'expérience.....	18
2. Quelques pistes de progrès.....	18

I. Description du projet

Le projet consiste à mettre en pratique les principaux concepts relatifs aux systèmes de bases de données NoSQL distribués tout en expérimentant l'un de ces systèmes en particulier. Le système retenu pour être l'objet de cette étude est **Riak**.

Dans le détail, les objectifs sont les suivants :

- Collecter les données devant servir de matière première à l'étude
- Mettre en œuvre le système Riak et constituer plusieurs collections de données puis expérimenter le système et ses fonctionnalités basiques
- Approfondir un ou plusieurs aspects de Riak
- Créer une base de travail pour un futur projet d'analyse Big Data

II. Les données

1. Fournisseurs et structures des données

La philosophie d'ouverture des données (open data) étant de plus en plus répandue, les fournisseurs de données se multiplient, que leur activité soit de nature publique ou privée. Le plus souvent, ces données sont représentées sous forme sérialisée, au format XML ou JSON. Généralement, les fournisseurs mettent à disposition des interfaces REST, des web services permettant d'envoyer des requêtes http (dans ce contexte, exclusivement des requêtes de type GET).

Parmi tous les fournisseurs, nous citerons deux exemples :

- **openweathermap.org** fournit des données relatives aux relevés météorologiques partout dans le monde. Ces relevés peuvent correspondre à des données d'historique, aux informations actuelles, ou encore aux prévisions à court et moyen terme. Dans notre étude, nous nous intéressons aux relevés des données concernant la ville de Paris à un instant T. Pour les obtenir, on exécutera la requête suivante :

```
curl -X GET http://api.openweathermap.org/data/2.5/weather?q=Paris,fr
```

Le message JSON retourné est de la forme suivante :

```
{"coord":{"lon":2.35,"lat":48.85},  
  "weather":[{"id":804,"main":"Clouds","description": "overcast  
clouds","icon":"04d"}],  
  "base":"stations",  
  "main":{"temp":292.11,"pressure":1017,"humidity":52,  
    "temp_min":290.93,"temp_max":293.15},  
  "visibility":10000,  
  "wind":{"speed":7.2,"deg":280},  
  "clouds":{"all":90},  
  "dt":1436361212,  
  "sys":{"type":1,"id":5615,"message":0.0094,"country":"FR","su  
nrise":1436327793,"sunset":1436385264},  
  "id":2988507,"name":"Paris","cod":200}
```

On remarque que ce document ne mentionne pas la date du relevé météo. Il faudra donc faire apparaître la date/heure dans le nom du fichier JSON sauvegardé si l'on souhaite conserver cette information.

- Autre exemple, **JCDecaux.com** offre un service complet d'open data concernant ses différentes activités comme par exemple des informations en temps réel sur le service Vélib à Paris. Ici aussi, c'est via une interface REST que l'on communique avec les web services. A noter qu'une clé API est requise pour accéder aux données. Pour l'obtenir, une inscription (gratuite) est nécessaire à l'adresse ci-dessous :

<https://developer.jcdecaux.com/#/opendata/vls?page=getstarted>

Les premières données intéressantes à collecter sont les **contrats**. Chez JCDecaux, un contrat est caractérisé par un nom (généralement la plus grande ville concernée par le contrat), un nom commercial, un pays, et la liste des communes incluses dans le contrat. Pour obtenir la liste des contrats, on exécute la requête ci-dessous :

```
curl -X GET https://api.jcdecaux.com/vls/v1/contracts?apiKey=Key
```

Cette requête renvoie un document JSON sous la forme suivante :

```
[{"name": "Rouen",
  "cities": ["Rouen"],
  "commercial_name": "cy'clik",
  "country_code": "FR"},
 {"name": "Paris",
  "cities": ["Arcueil", "Aubervilliers", "Bagnolet", "Boulogne
  Billancourt", "Charenton", "Clichy", "Fontenay-sous-
  Bois", "Gentilly", "Issy les Moulineaux", "Ivry", "Joinville", "Le
  Kremlin Bicêtre", "Le Pré St Gervais", "Les Lilas", "Levallois-
  Perret", "Malakoff", "Montreuil", "Montrouge", "Neuilly", "Nogent"
  , "Pantin", "Paris", "Puteaux", "Saint Cloud", "Saint
  Denis", "Saint Mandé", "Saint Maurice", "Saint
  Ouen", "Suresnes", "Vanves", "Vincennes"],
  "commercial_name": "Vélib",
  "country_code": "FR"}, ...]
```

Il s'agit d'un tableau de valeurs complexes représentant tous les contrats. On remarque ainsi que le contrat qui nous intéresse s'appelle « Paris ». Les communes rattachées à ce contrat sont toutes énumérées. Pour obtenir les informations détaillées sur une ou plusieurs stations, deux possibilités sont offertes. La première consiste à récupérer les informations de toutes les stations du contrat, en un seul gros document JSON, de cette façon :

```
curl -X GET
```

```
"https://api.jcdecaux.com/vls/v1/stations?contract=Paris&apiKey=Key"
```

On peut également interroger une station en particulier. Pour cela, il faut connaître l'identifiant de la station en question, ce qui nécessite d'avoir interrogé au moins une fois le service de manière globale afin de récupérer tous les identifiants. C'est l'option que nous avons privilégiée dans notre étude. Elle nous permet d'avoir un fichier différencié pour chaque station. La requête à exécuter est la suivante (id est l'identifiant station) :

```
curl -X GET
```

```
https://api.jcdecaux.com/vls/v1/stations/id?contract=Paris&apiKey=Key
```

Dans tous les cas, les données renvoyées sont sous cette forme :

```
{"number":11008,  
  "name":"11008 - FAIDHERBE CHALIGNY",  
  "address":"223 RUE DU FAUBOURG SAINT ANTOINE - 75011 PARIS",  
  "position":{"lat":48.850379753879935,"lng":2.38386933547444},  
  "banking":true,  
  "bonus":false,  
  "status":"OPEN",  
  "contract_name":"Paris",  
  "bike_stands":40,  
  "available_bike_stands":1,  
  "available_bikes":39,  
  "last_update":1434878133000}
```

Le service Vélib dispose de 1230 stations en tout. Par conséquent, la requête globale renvoie un tableau de 1230 valeurs complexes représentant chacune une station.

2. Collecte et volumétrie

Le but de cette étude étant de travailler sur un volume de données suffisamment conséquent pour rendre l'expérience intéressante, il était inenvisageable de réaliser manuellement la collecte de toutes les données.

Afin d'automatiser le processus, mon choix s'est porté sur un nano ordinateur Raspberry Pi, dont les performances sont largement suffisantes pour ce genre de traitements et dont la particularité lui permet de rester allumé et connecté en continu pendant une période assez longue.

Deux scripts ont ainsi été écrit :

- l'un pour les données avec en sortie un fichier JSON
- l'autre pour les données Vélib avec en sortie 1230 fichiers JSON (un par station)

Chaque script sera exécuté toutes les heures. Si une erreur survient pendant l'exécution (service inaccessible, panne internet, etc.), un fichier JSON sera tout de même généré, à vide.

L'objectif à long terme de cette collecte est de réaliser une analyse approfondie sur un gros volume de données afin de rechercher une éventuelle corrélation entre le niveau de disponibilité du service Vélib et l'un ou plusieurs des critères suivants :

- La période de l'année
- Le moment de la journée
- La localisation géographique
- La météo du moment

Néanmoins, dans le cadre de la présente étude, nous limiterons aux données de 7 jours, ce qui représente tout de même :

- $24 \times 7 = 168$ fichiers pour la météo
- $24 \times 7 \times 1230 = 206\,640$ fichiers pour Vélib

Chaque fichier pesant en moyenne 1Ko, il est facile d'évaluer le poids global de cette collecte (environ 200Mo pour 1 semaine de données).

Naturellement, ce genre de processus automatisé étant sujet à de probables perturbations voire interruptions, une surveillance a également été mise en place. Elle consiste en l'envoi régulier (toutes les heures avec un décalage d'une demi-heure sur les scripts de collecte) d'un email indiquant d'une part que le Raspberry Pi est en ligne et d'autre part où en est la collecte (nombre de fichiers déjà collectés). Le message envoyé est sous cette forme :

Date: 25 juin 2015 07:30
Objet: Raspberry PI
Envoyé par: gmail.com

Hello, I am alive! :)
86 files for meteo
105780 files for stations

A l'issue de l'étape de collecte, le bilan suivant a pu être effectué :

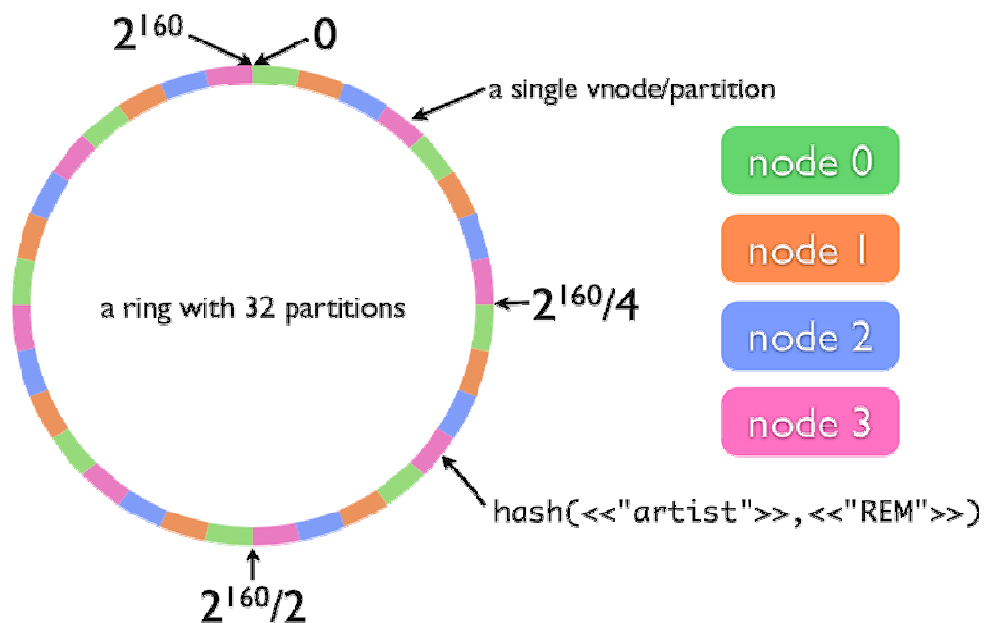
- Météo : 168 fichiers collectés, dont 0 en échec
- Vélib : 206 640 fichiers collectés, dont 174 en échec (vides)

III. Le système NoSQL Riak

1. Présentation de Riak

Riak est un système NoSQL, développé en langage Erlang, fonctionnant sur le modèle des paires clé-valeur. En mode distribué, Riak fonctionne selon une architecture multi-nœuds, c'est-à-dire sans **nœud maître**. Tous les nœuds d'un cluster Riak sont égaux. Le client peut donc effectuer une opération de lecture ou d'écriture avec n'importe quel nœud du cluster. L'avantage immédiat d'une telle architecture est l'absence de point individuel de défaillance (ou SPOF pour Single Point Of Failure), ce qui rend le système moins vulnérable aux pannes. L'inconvénient est en revanche un risque important concernant la cohérence des données.

Riak utilise un mode de partitionnement par hachage consistant. Il est organisé en anneau dont la taille est fixée de façon définitive au départ. Les nœuds physiques sont ensuite répartis équitablement sur l'anneau. En réalité, chaque nœud physique, « héberge » des nœuds virtuels, appelés vnodes, et ces vnodes sont en charge chacun d'une partition de l'anneau. Dans l'exemple ci-dessous, nous avons un anneau de taille 32 et 4 nœuds physiques. Chaque nœud physique hébergera donc 8 vnodes, pour un total de 32 vnodes en charge des 32 partitions de l'anneau.



Source : docs.basho.com

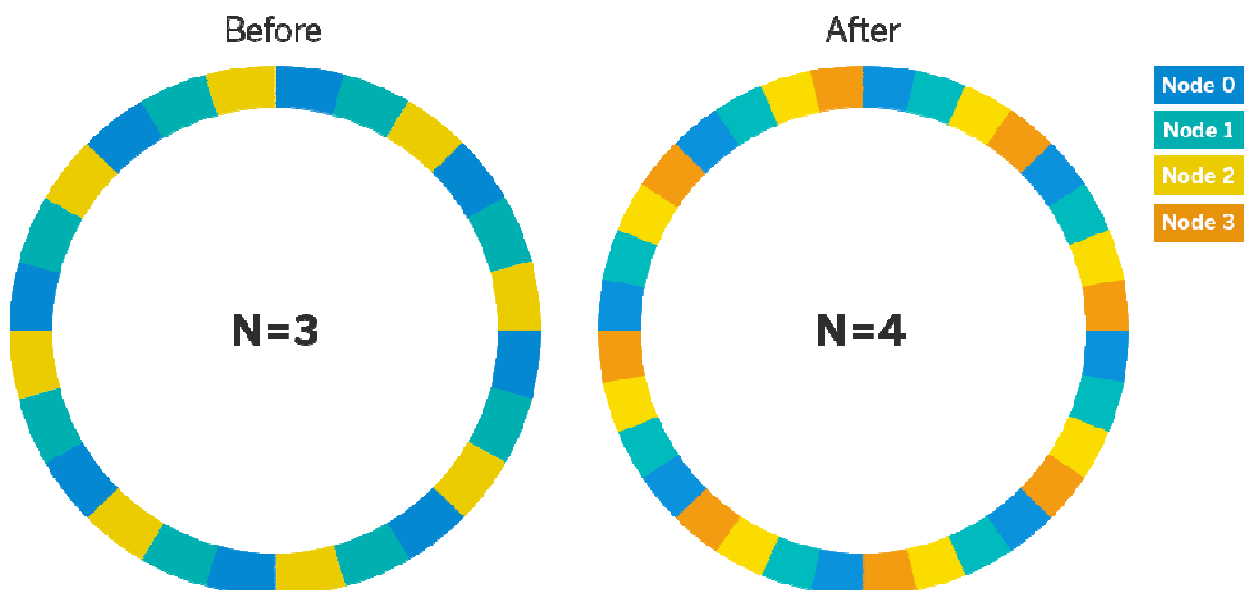
Riak propose en outre une réplication des données, grâce à la variable `n_val` (dont la valeur par défaut est 3 et dont la valeur maximale est égale au nombre de nœuds physiques). Concrètement, pour une valeur de `n_val = 3`, 3 copies de chaque document seront stockées. Généralement, lorsqu'un document est enregistré sur une vnode d'un nœud physique N, les réplicas sont stockés sur une vnode des nœuds physiques qui suivent N sur l'anneau.

Riak introduit également une notion importante : le bucket. Le bucket se comporte comme un sous ensemble de paires clé-valeur, pouvant avoir un lien entre elles. Si nous faisons l'analogie avec les SGBDR, le bucket pourrait être vu comme une table.

Riak fonctionne historiquement dans un mode assurant une grande disponibilité et une forte tolérance aux pannes, ce qui, par application du théorème CAP le rend moins efficace en termes de cohérence des données. Riak offre généralement une cohérence à terme.

Pour résoudre les conflits d'écriture et/ou de mises à jour, Riak propose deux options : **le dernier qui écrit gagne** ou **renvoi des deux versions au client qui résoudra le conflit lui-même**.

Lors de l'ajout d'un nœud physique au sein d'un cluster Riak, l'anneau est réorganisé et l'attribution des partitions modifiée. Ceci est illustré sur la figure ci-dessous, avec le passage d'un cluster de 3 à 4 nœuds physiques :



Source : <http://blog.softlayer.com>

Pour terminer, il est à noter que Riak offre nativement des fonctionnalités MapReduce et fournit également un moteur de recherche plein-texte grâce à Riak Search, qui utilise notamment le moteur d'Apache Solr.

2. Mise en œuvre

a. Installation

Dans cette étude, nous avons installé **Riak 2.1.1** pour **Linux Ubuntu 14.04**, à partir du package disponible ici : <http://docs.basho.com/riak/latest/downloads/>

L'installation est très rapide et automatisée.

b. Test des fonctionnalités de base

Dès la fin de l'installation, on peut réaliser quelques tests en exécutant les commandes de base, sans même modifier (pour le moment) le fichier de configuration. Par exemple :

Pour vérifier l'installation et la configuration, on peut exécuter la commande ci-dessous (à noter que toutes les commandes doivent être exécutées en tant que superadmin) :

sudo riak chkconfig retourne :

config is OK

```
-config /var/lib/riak/generated.configs/app.2015.07.07.21.57.32.config -args_file  
/var/lib/riak/generated.configs/vm.2015.07.07.21.57.32.args -vm_args  
/var/lib/riak/generated.configs/vm.2015.07.07.21.57.32.args
```

Pour démarrer un nœud, la commande *sudo riak start* ne retourne rien si tout se passe bien mais indique un message d'erreur si le nœud n'a pas pu démarrer correctement.

Une fois démarré, on peut interroger le statut du nœud par la commande *sudo riak-admin cluster status* qui retourne un résultat du type :

---- Cluster Status ----

Ring ready: true

```
+-----+-----+-----+-----+-----+
|           node           |status| avail |ring |pending|
+-----+-----+-----+-----+-----+
| (C) riak@127.0.0.1       |valid |  up   |100.0|  --   |
+-----+-----+-----+-----+-----+
```

Key: (C) = Claimant; availability marked with '!' is unexpected

Le nœud étant bien fonctionnel, nous pouvons passer aux commandes REST. A noter que lors de l'envoi d'une requête PUT, la création du bucket est automatique.

```
curl -X PUT 'http://localhost:8098/riak/tests/test1' -H 'Content-Type:text/plain' -d
'this is the first test'
```

Dans la requête ci-dessus, le bucket est **tests**, la clé **test1**, et la valeur **this is the first test**.

La commande *curl -X GET 'http://localhost:8098/riak/tests/test1'* retourne donc **this is the first test**.

Pour supprimer la clé **test1**, on exécute la commande *curl -X DELETE 'http://localhost:8098/riak/tests/test1'*

Ainsi, si on relance *curl -X GET 'http://localhost:8098/riak/tests/test1'*, on obtient : **not found**.

Essayons maintenant des données plus complexes en injectant un des fichiers JSON collectés précédemment, avec la commande suivante :

```
curl -i -X PUT http://localhost:8098/riak/meteo/paris_20150621_180001 -H "Content-
Type: application/json" -d
'{"coord":{"lon":2.35,"lat":48.85},"sys":{"message":0.0383,"country":"FR","sunrise":143
4858417,"sunset":1434916675},"weather":[{"id":803,"main":"Clouds","description":"bro
```

```
ken clouds","icon":"04d"}], "base":"stations","main":{"temp":295.52,"temp_min":295.52,
"temp_max":295.52,"pressure":1019.06,"sea_level":1031.06,"grnd_level":1019.06,"humi
dity":55},"wind":{"speed":5.82,"deg":289.503},"clouds":{"all":64},"dt":1434902407,"id":
2988507,"name":"Paris","cod":200}'
```

La commande `curl -i -X GET http://localhost:8098/riak/meteo/paris_20150621_180001` retourne alors :

```
{"coord":{"lon":2.35,"lat":48.85},"sys":{"message":0.0383,"country":"FR","sunris
e":1434858417,"sunset":1434916675},"weather":[{"id":803,"main":"Clouds","desc
ription":"broken clouds","icon":"04d"}],
"base":"stations","main":{"temp":295.52,"temp_min":295.52,
"temp_max":295.52,"pressure":1019.06,"sea_level":1031.06,"grnd_level":1019.06,"
humidity":55},"wind":{"speed":5.82,"deg":289.503},"clouds":{"all":64},"dt":14349
02407,"id":2988507,"name":"Paris","cod":200}
```

Notre nœud Riak semble donc bel et bien fonctionnel.

3. Expérimentations & approfondissement

a. Construction d'un mini-cluster (2 nœuds)

Pour expérimenter le mode distribué, nous allons prendre deux machines physiques distinctes, dont les adresses IP sont **192.168.0.100** et **192.168.0.101** il faut modifier le fichier de configuration `/etc/riak/riak.conf` sur les 2 postes. L'option `n_val` sera ici paramétrée à 2.

Dans ce fichier, nous pouvons trouver entre autres les options suivantes :

```
#nombre de partitions dans le cluster
ring_size = 64
```

```
#active l'interface web d'administration
riak_control = on
```

Nous modifierons les entrées ci-dessous :

```
Sur le 1er nœud
nodename = riak@192.168.0.100
listener.protobuf.internal = 192.168.0.100:8087
listener.http.internal = 192.168.0.100:8098
```

```
Sur le 2nd nœud
nodename = riak@192.168.0.101
listener.protobuf.internal = 192.168.0.101:8087
listener.http.internal = 192.168.0.101:8098
```

Exécutons maintenant la commande *sudo riak start* sur chacun de nos deux nœuds physiques. Puis, lançons la commande *sudo riak-admin cluster status* sur chaque machine. Nous obtenons les résultats ci-dessous :

---- Cluster Status ----

Ring ready: true

```
+-----+-----+-----+-----+
|      node      |status| avail |ring |pending|
+-----+-----+-----+-----+
| (C) riak@192.168.0.100 |valid |  up   |100.0|  --   |
+-----+-----+-----+-----+
```

Key: (C) = Claimant; availability marked with '!' is unexpected

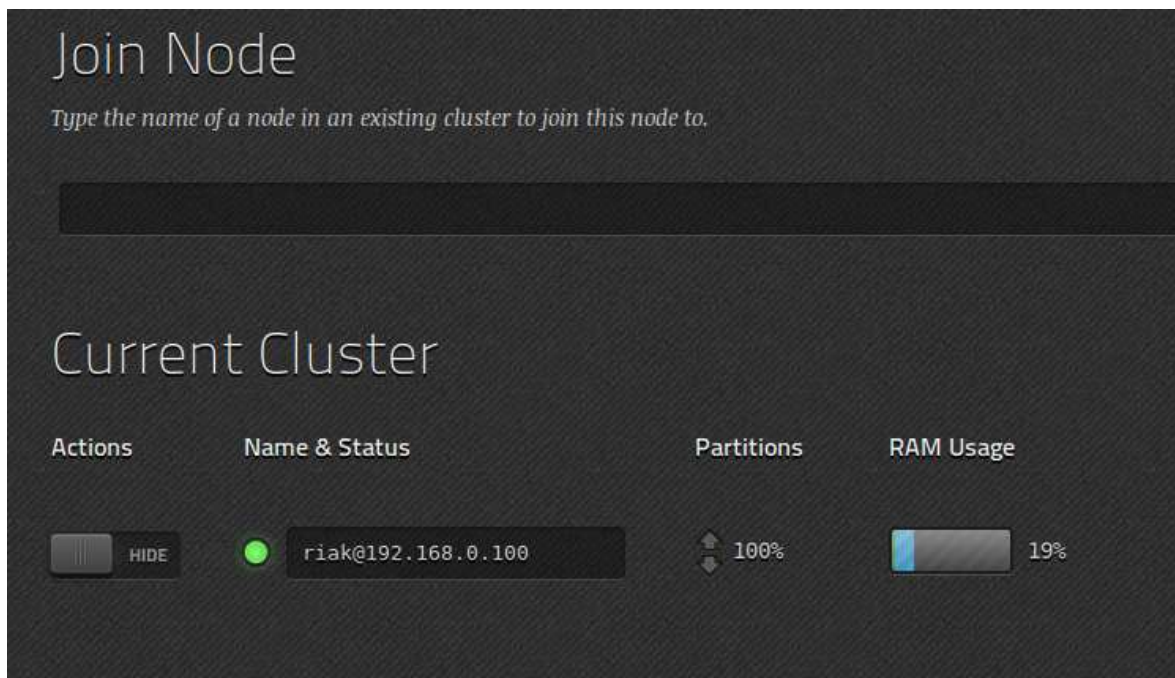
---- Cluster Status ----

Ring ready: true

```
+-----+-----+-----+-----+
|      node      |status| avail |ring |pending|
+-----+-----+-----+-----+
| (C) riak@192.168.0.101 |valid |  up   |100.0|  --   |
+-----+-----+-----+-----+
```

Key: (C) = Claimant; availability marked with '!' is unexpected

L'interface web, disponible à : <http://192.168.0.100:8098/adminin/cluster> nous permet de constater que le nœud est fonctionnel :



Il apparait que nos deux nœuds fonctionnent comme deux clusters indépendants. Pour le vérifier, nous allons stocker un document sur le premier nœud :

```
curl -X PUT 'http://192.168.1.100:8098/riak/tests/test1' -H 'Content-Type:text/plain' -d 'this is the first test'
```

Puis, essayons une lecture de ce document sur chaque nœud :

```
curl -X GET 'http://192.168.0.100:8098/riak/tests/test1' retourne this is a test
```

MAIS

```
curl -X GET 'http://192.168.0.101:8098/riak/tests/test1' retourne not found
```

Pour joindre le nœud 2 au nœud 1, on exécutera la commande suivante :

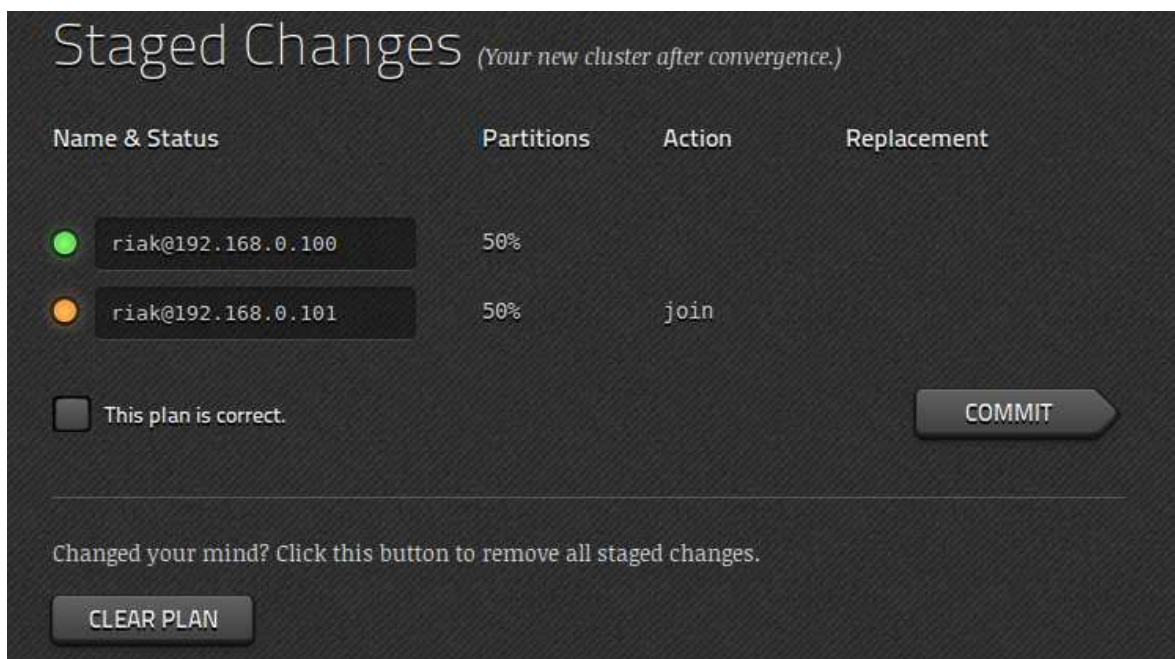
```
sudo riak-admin cluster join riak@192.168.0.100
```

suivie de `sudo riak-admin plan`

L'interface web nous montre bien l'intention du nœud 2 de rejoindre le cluster :



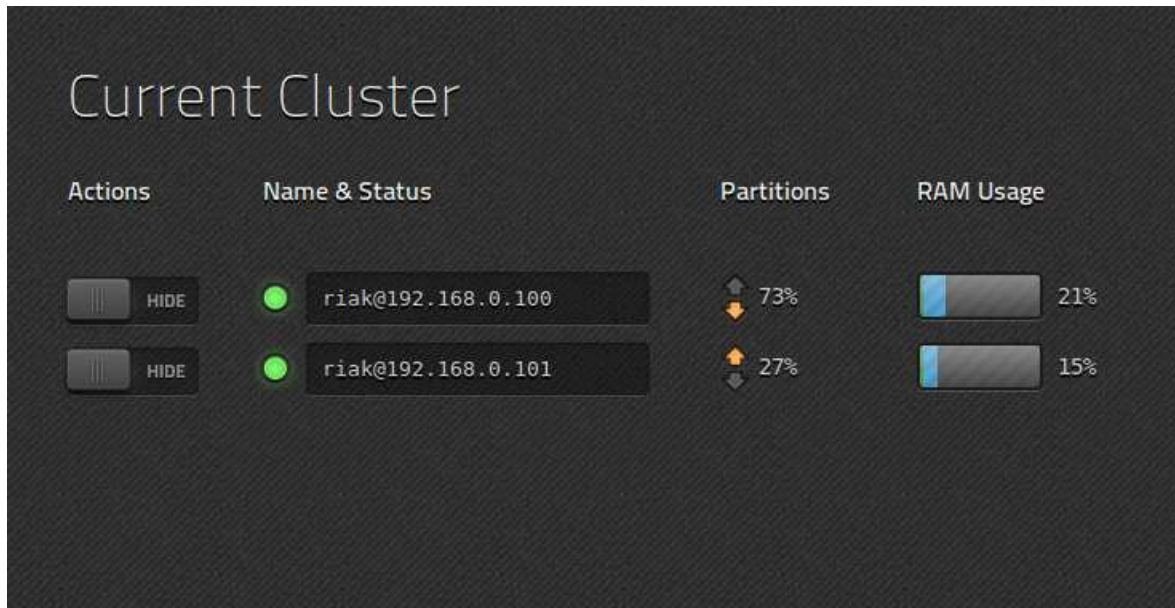
Et nous donne une représentation prévisible du cluster après confirmation de l'opération, à savoir une répartition égale sur l'anneau :



La commande `sudo riak-admin cluster commit` confirme les changements.

Elle renvoie la réponse **Cluster changes committed**

Dès lors, le nœud 2 passe au vert et le transfert des partitions du nœud 1 vers le nœud 2 commence (voir ci-dessous) :



A la fin du processus, la commande `sudo riak-admin cluster status` exécutée sur n'importe quel nœud donne le résultat ci-dessous :

---- Cluster Status ----

Ring ready: true

```
+-----+-----+-----+-----+-----+
|           node           |status| avail |ring |pending|
+-----+-----+-----+-----+-----+
| (C) riak@192.168.0.100 |valid |  up   | 50.0|  --   |
|   riak@192.168.0.101 |valid |  up   | 50.0|  --   |
+-----+-----+-----+-----+-----+
```

Key: (C) = Claimant; availability marked with '!' is unexpected

A présent nous réessayons une lecture sur chaque nœud :

`curl -X GET 'http://192.168.0.100:8098/riak/tests/test1'` retourne **this is a test**

`curl -X GET 'http://192.168.0.101:8098/riak/tests/test1'` retourne **this is a test**

Le cluster fonctionne, la réplication aussi.

b. Constitution d'une collection de données

Disposant de plus de 200 000 fichiers JSON, les intégrer manuellement dans Riak n'est pas envisageable. Là encore, il a été nécessaire de créer un script pour automatiser le processus.

La stratégie retenue est la suivante : nous allons créer 2 buckets (meteo et velib). A l'intérieur de chaque bucket, une clé donnée correspondra à un fichier JSON.

Une clé meteo sera de la forme : Paris_aaaammjj_hhmmss

Une clé Vélib sera de la forme : IdStation_aaaammjj_hhmmss

Pour chaque clé, la valeur est le contenu du fichier JSON collecté.

Cette insertion en masse a été exécutée sur le cluster distribué avec les 2 nœuds en ligne et a duré environ 2 heures.

c. Riak Search (Solr)

Riak propose un moteur de recherche indexé, offrant des possibilités de recherche plein-texte. Cette fonctionnalité est basée sur le moteur de Solr. Afin d'utiliser Riak Search, un certain nombre d'options doivent être modifiées dans le fichier de configuration :

`search.root_dir` indique l'emplacement de stockage des index

`search.solr.start_timeout` correspond au temps d'attente avant que Riak ne considère une requête comme échouée.

`search.solr.port` spécifie le numéro de port de fonctionnement de Solr

`search` est enfin l'option la plus importante. Elle doit être configurée à **on** pour que Riak Search soit activé.

d. MapReduce avec Riak

Riak implémente nativement un moteur MapReduce. Les fonctions peuvent être écrites en Erlang ou Javascript. Dans cette étude, nous n'avons malheureusement pas expérimenté cette fonctionnalité en détail.

IV. Conclusion

1. Bilan de l'expérience

Riak apparaît comme un système distribué offrant des performances très satisfaisantes, tant pour un mode d'utilisation orienté système temps réel, notamment grâce à l'implémentation de Riak Search (basé sur Solr), que pour des traitements analytiques, grâce à ses fonctionnalités MapReduce.

Riak est par ailleurs assez simple à administrer et offre un très bon niveau de disponibilité et une très grande tolérance aux pannes. En termes de cohérence de données, il tend également à s'améliorer sensiblement dans sa dernière version.

Naturellement, comme tous les systèmes distribués, Riak ne prend sa pleine mesure, tant dans ses performances que dans sa robustesse, qu'à partir d'un nombre important de nœuds physiques.

2. Quelques pistes de progrès

Cette étude nous a permis d'aborder le fonctionnement global de Riak et d'en approfondir certains aspects. Pour aller plus loin, nous pouvons envisager deux pistes principales .

- Mettre en service un cluster avec un nombre de nœuds physiques plus important. Les serveurs présents dans les grappes de serveurs des systèmes distribués sont appelés *commodity servers* (c'est-à-dire peu coûteux, des serveurs que l'on remplace plutôt qu'on ne les répare en cas de défaillance). Le Raspberry Pi étant un parfait candidat au rôle de commodity server, il serait intéressant de construire un cluster de 5 à 10 nœuds physiques en utilisant ce type de matériel et ainsi pouvoir pousser plus loin les tests relatifs au partitionnement et aux tolérances aux pannes.
- Même si les données utilisées ici étaient déjà conséquentes, se projeter vers un horizon Big Data en collectant les données pendant un an ou plus permettrait d'éprouver un peu plus Riak, tant pour la partie temps réel qu'analytique.

- Enfin, cette étude n'ayant été qu'une première découverte de Riak, il me reste à approfondir certain aspects et aller un peu plus loin notamment avec Riak Search et MapReduce.